

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics

Practical – VIII

Roll No.: T038	Name: Akansha Pritam Ranade
Class: TYIT	Batch: 1
Date of Assignment: 11-09-2026	Date/Time of Submission: 11-09-2026

Part A: Confidence Interval Estimation

1. Load the supermarket_sales.csv dataset into a Pandas DataFrame.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
df = pd.read_csv("SuperMarket Analysis.csv")
```

2. Display the first five records and check the size of the dataset.

```
print("First 5 records:")
display(df.head())
print("Dataset Size:", df.shape)
print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])
print("Akansha T038")
```

First 5 records:

	Invoice ID	Branch	City	Customer type	Gender	
0	750-67-8428	Alex	Yangon	Member	Female	H
1	226-31-3081	Giza	Naypyitaw	Normal	Female	B ac
2	631-41-3108	Alex	Yangon	Normal	Female	H
3	123-19-1176	Alex	Yangon	Member	Female	H

```
Dataset Size: (1000, 17)
Number of Rows: 1000
Number of Columns: 17
Akansha T038
```

3. Select the revenue column for analysis.

```
revenue = df['Sales']
print("Revenue column selected:")
display(revenue.head())
print("Akansha T038")
```

```
Revenue column selected:

0    548.9715
1     80.2200
2    340.5255
3    489.0480
4    634.3785
Name: Sales, dtype: float64

Akansha T038
```

4. Check and handle missing values in the selected column.

```
print("Missing values:", revenue.isnull().sum())
revenue = revenue.dropna()
print("Missing values after handling:", revenue.isnull().sum())
print("Akansha T038")
```

```
Missing values: 0
Missing values after handling: 0
Akansha T038
```

5. Calculate the sample size, sample mean, and sample standard deviation.

```
sample_size = len(revenue)
sample_mean = revenue.mean()
sample_std = revenue.std()
print("Sample Size:", sample_size)
print("Sample Mean:", round(sample_mean, 2))
print("Sample Standard Deviation:", round(sample_std, 2))
print("Akansha T038")
```

```
Sample Size: 1000
Sample Mean: 322.97
Sample Standard Deviation: 245.89
Akansha T038
```

6. Calculate the 90% confidence interval for the population mean revenue.

7. Calculate the 95% confidence interval for the population mean revenue.

8. Calculate the 99% confidence interval for the population mean revenue.

```
confidence_levels = [0.90, 0.95, 0.99]
```

```
results = []
```

```

for confidence in confidence_levels:
    alpha = 1 - confidence

    t_critical = stats.t.ppf(
        1 - alpha / 2,
        df=sample_size - 1
    )

    margin_error = t_critical * (
        sample_std / np.sqrt(sample_size)
    )

    lower = sample_mean - margin_error
    upper = sample_mean + margin_error

    results.append([
        int(confidence * 100),
        lower,
        upper,
        margin_error
    ])

```

9. Display the confidence intervals in a suitable table.

```

ci_table = pd.DataFrame(
    results,
    columns=[
        'Confidence Level',
        'Lower Bound',
        'Upper Bound',
        'Margin of Error'
    ]
)

display(ci_table.round(2))
print("Akansha T038")

```

	Confidence Level	Lower Bound	Upper Bound	Margin of Error
0	90	310.17	335.77	12.80
1	95	307.71	338.23	15.26
2	99	302.90	343.03	20.07

Akansha T038

10. Compare the 90%, 95%, and 99% confidence intervals.

```
print("Comparison of Confidence Intervals\n")
```

```
for _, row in ci_table.iterrows():
```

```
    print(
        f"{int(row['Confidence Level'])}% CI: "
        f"{row['Lower Bound']:.2f} to "
        f"{row['Upper Bound']:.2f}"
    )
```

```
print("\nObservation:")
```

```
print("90% confidence interval is the narrowest.")
```

```
print("95% confidence interval is wider than the 90% interval.")
```

```
print("99% confidence interval is the widest.")
```

```
print("As the confidence level increases, the interval becomes wider.")
```

```
print("Akansha T038")
```

```
Comparison of Confidence Intervals

90% CI: 310.17 to 335.77
95% CI: 307.71 to 338.23
99% CI: 302.90 to 343.03

Observation:
90% confidence interval is the narrowest.
95% confidence interval is wider than the 90% interval.
99% confidence interval is the widest.
As the confidence level increases, the interval becomes wider
Akansha T038
```

11. Create a suitable visualization to display the sample mean and confidence interval.

```
plt.figure(figsize=(8, 5))
```

```

levels = ci_table['Confidence Level']
lower = ci_table['Lower Bound']
upper = ci_table['Upper Bound']

for i in range(len(ci_table)):

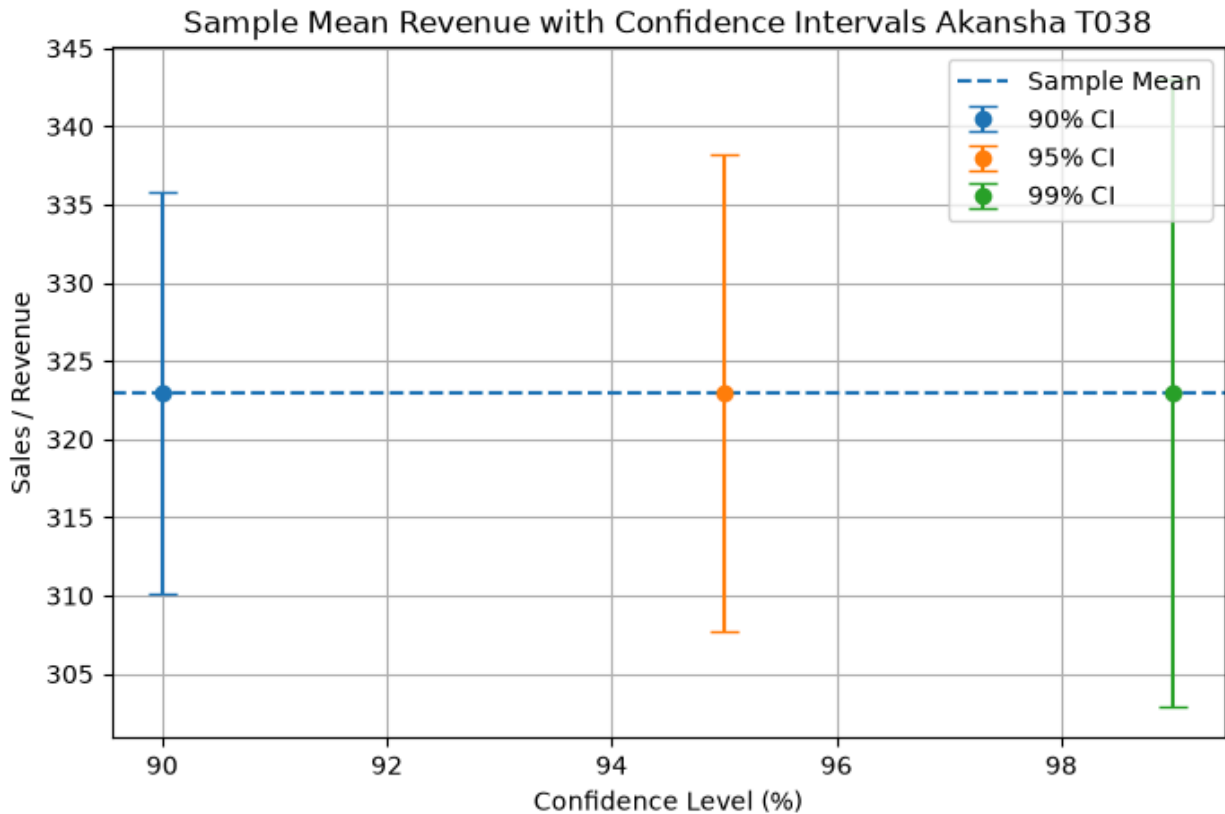
    plt.errorbar(
        levels.iloc[i],
        sample_mean,
        yerr=[
            [sample_mean - lower.iloc[i]],
            [upper.iloc[i] - sample_mean]
        ],
        fmt='o',
        capsize=6,
        label=f"{int(levels.iloc[i])}% CI"
    )

plt.axhline(
    sample_mean,
    linestyle='--',
    label='Sample Mean'
)

plt.xlabel("Confidence Level (%)")
plt.ylabel("Sales / Revenue")
plt.title("Sample Mean Revenue with Confidence Intervals Akansha T038")
plt.legend()
plt.grid(True)

plt.show()

```



12. Interpret the 95% confidence interval in the context of supermarket revenue.

```
lower_95 = ci_table.loc[
    ci_table['Confidence Level'] == 95,
    'Lower Bound'
].iloc[0]
```

```
upper_95 = ci_table.loc[
    ci_table['Confidence Level'] == 95,
    'Upper Bound'
].iloc[0]
```

```
print(
    f"95% Confidence Interval: "
    f"{lower_95:.2f} to {upper_95:.2f}"
)
print("Akansha T038")
```

```
95% Confidence Interval: 307.71 to 338.23
Akansha T038
```

13. Select the customer_type column and calculate the proportion of customers belonging to each category.

```
customer_counts = df['Customer type'].value_counts()
```

```

customer_proportion = df['Customer type'].value_counts(
    normalize=True
)
customer_table = pd.DataFrame({
    'Count': customer_counts,
    'Proportion': customer_proportion
})
customer_table['Percentage'] = (
    customer_table['Proportion'] * 100
)
display(customer_table.round(2))
print("Akansha T038")

```

	Count	Proportion	Percentage
Customer type			
Member	565	0.56	56.5
Normal	435	0.44	43.5

Akansha T038

14. Calculate the 95% confidence interval for the proportion of Member customers.

```

member_count = (
    df['Customer type'] == 'Member'
).sum()

n_customers = df['Customer type'].notna().sum()

member_proportion = member_count / n_customers

z_critical = stats.norm.ppf(0.975)

margin_error = z_critical * np.sqrt(
    (member_proportion * (1 - member_proportion))
    / n_customers
)

lower_member = member_proportion - margin_error
upper_member = member_proportion + margin_error

print("Member Count:", member_count)
print(

```

```

"Member Proportion:",
round(member_proportion, 4)
)

print(
    f"95% Confidence Interval: "
    f"{lower_member:.4f} to {upper_member:.4f}"
)

print(
    f"Percentage: "
    f"{lower_member*100:.2f}% to "
    f"{upper_member*100:.2f}%"
)
print("Akansha T038")

```

```

Member Count: 565
Member Proportion: 0.565
95% Confidence Interval: 0.5343 to 0.5957
Percentage: 53.43% to 59.57%
Akansha T038

```

15. Interpret the confidence interval for the population proportion in a business context.

We are 95% confident that the true population proportion of Member customers lies between the calculated lower and upper confidence limits. This information can help the supermarket understand its customer loyalty base and plan membership offers, promotions and customer-retention strategies.

Part B: Bootstrapping

16. Select the revenue column and calculate its original sample median.

```
original_median = revenue.median()
```

```

print(
    "Original Sample Median Revenue:",
    round(original_median, 2)
)
print("Akansha T038")

```

```

Original Sample Median Revenue: 253.85
Akansha T038

```

17. Create a bootstrap sample by randomly selecting observations from the original sample with replacement.

18. Calculate the median of the bootstrap sample.

```
bootstrap_sample = np.random.choice(
```

```

    revenue,
    size=len(revenue),
    replace=True
)
bootstrap_median = np.median(bootstrap_sample)
print(
    "Bootstrap Sample Median:",
    round(bootstrap_median, 2)
)
print("Akansha T038")

```

```

Bootstrap Sample Median: 264.93
Akansha T038

```

19. Repeat the bootstrapping process 1,000 times and store the median from each bootstrap sample.

```

bootstrap_medians = []

for i in range(1000):

    bootstrap_sample = np.random.choice(
        revenue,
        size=len(revenue),
        replace=True
    )

    median = np.median(bootstrap_sample)

    bootstrap_medians.append(median)

bootstrap_medians = np.array(bootstrap_medians)

print(
    "Number of bootstrap samples:",
    len(bootstrap_medians)
)
print("Akansha T038")

```

```

Number of bootstrap samples: 1000
Akansha T038

```

20. Create a histogram of the bootstrap median values.

```

plt.figure(figsize=(9, 5))

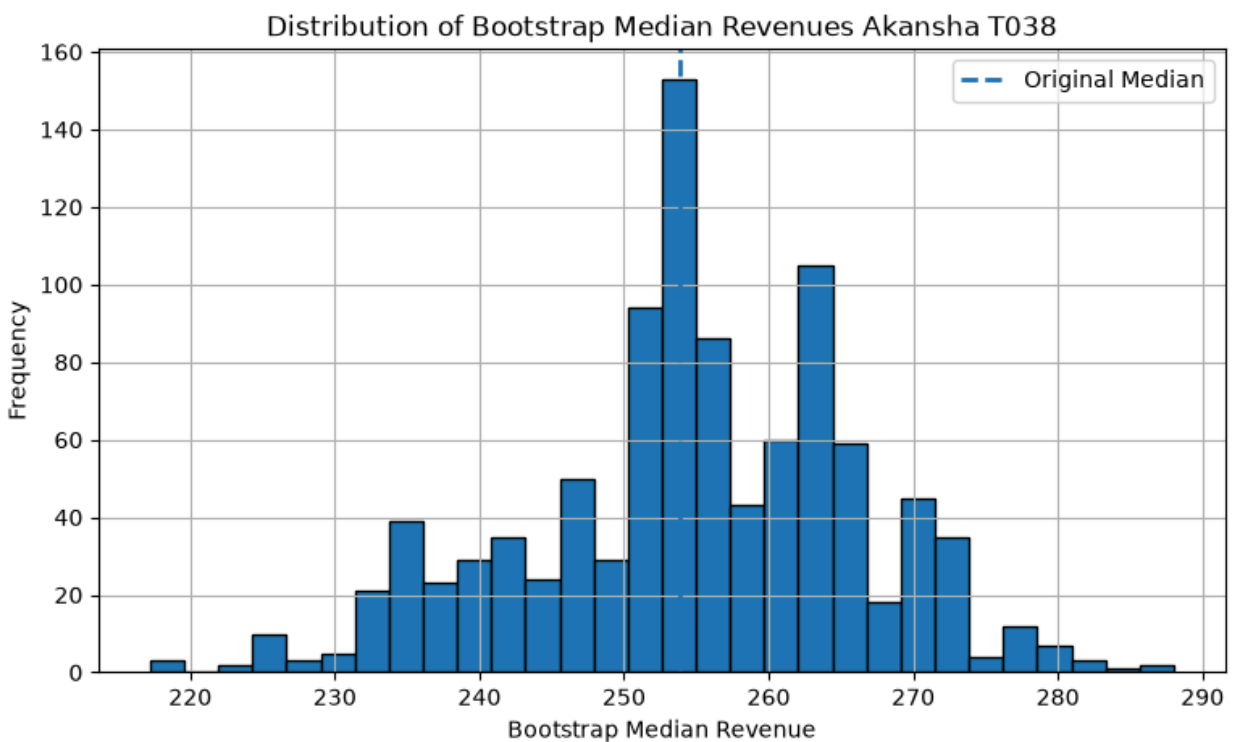
```

```
plt.hist(
    bootstrap_medians,
    bins=30,
    edgecolor='black'
)

plt.axvline(
    original_median,
    linestyle='--',
    linewidth=2,
    label='Original Median'
)

plt.xlabel("Bootstrap Median Revenue")
plt.ylabel("Frequency")
plt.title("Distribution of Bootstrap Median Revenues Akansha T038")
plt.legend()
plt.grid(True)

plt.show()
```



21. Calculate the 2.5th percentile and 97.5th percentile of the bootstrap median values.

```
lower_percentile = np.percentile(
    bootstrap_medians,
    2.5
```

```

)

upper_percentile = np.percentile(
    bootstrap_medians,
    97.5
)

print(
    "2.5th Percentile:",
    round(lower_percentile, 2)
)

print(
    "97.5th Percentile:",
    round(upper_percentile, 2)
)
print("Akansha T038")

```

```

2.5th Percentile: 232.39
97.5th Percentile: 275.41
Akansha T038

```

22. Use these percentile values to calculate the 95% bootstrap confidence interval for the population median.

```

print("95% Bootstrap Confidence Interval:")

print(
    f"({lower_percentile:.2f}, "
    f"{upper_percentile:.2f})"
)
print("Akansha T038")

```

```

95% Bootstrap Confidence Interval:
(232.39, 275.41)
Akansha T038

```

23. Display the original sample median and the bootstrap confidence interval.

```

summary_table = pd.DataFrame({
    'Statistic': [
        'Original Sample Median',
        '95% Bootstrap CI Lower',
        '95% Bootstrap CI Upper'
    ],
    'Value': [
        original_median,

```

```

        lower_percentile,
        upper_percentile
    ]
})

display(summary_table.round(2))
print("Akansha T038")

```

	Statistic	Value
0	Original Sample Median	253.85
1	95% Bootstrap CI Lower	232.39
2	95% Bootstrap CI Upper	275.41
Akansha T038		

24. Compare the original median with the bootstrap confidence interval.

```

print(
    "Original Sample Median:",
    round(original_median, 2)
)

print(
    "95% Bootstrap Confidence Interval:",
    f"{lower_percentile:.2f} "
    f"to {upper_percentile:.2f}"
)

if lower_percentile <= original_median <= upper_percentile:
    print(
        "\nThe original sample median lies "
        "within the bootstrap confidence interval."
    )
else:
    print(
        "\nThe original sample median lies "
        "outside the bootstrap confidence interval."
    )
print("Akansha T038")

```

Original Sample Median: 253.85

95% Bootstrap Confidence Interval: 232.39 to 275.41

The original sample median lies within the bootstrap confidence interval.

Akansha T038

25. Interpret the bootstrap confidence interval in the context of supermarket revenue.

We are 95% confident that the true population median revenue lies between the lower and upper bootstrap confidence limits. The original sample median lies within this interval, indicating that the sample median is reasonably consistent with the estimated population median.

26. Write a conclusion explaining how confidence intervals and bootstrapping can help businesses make decisions using sample data.

Conclusion: Confidence intervals and bootstrapping are useful statistical techniques for making business decisions using sample data. Confidence intervals provide a range of plausible values for population parameters such as mean revenue and customer proportions.

The 90%, 95% and 99% confidence intervals show that increasing the confidence level produces a wider interval. This helps businesses understand the uncertainty associated with their estimates.

Bootstrapping estimates the uncertainty of statistics such as the median by repeatedly creating samples with replacement. The 95% bootstrap confidence interval provides a useful range for estimating the population median revenue.

For a supermarket, these techniques can support decisions related to revenue forecasting, pricing, promotions, inventory planning and customer management. Therefore, confidence intervals and bootstrapping help businesses make more reliable and data-driven decisions instead of relying only on a single sample value.